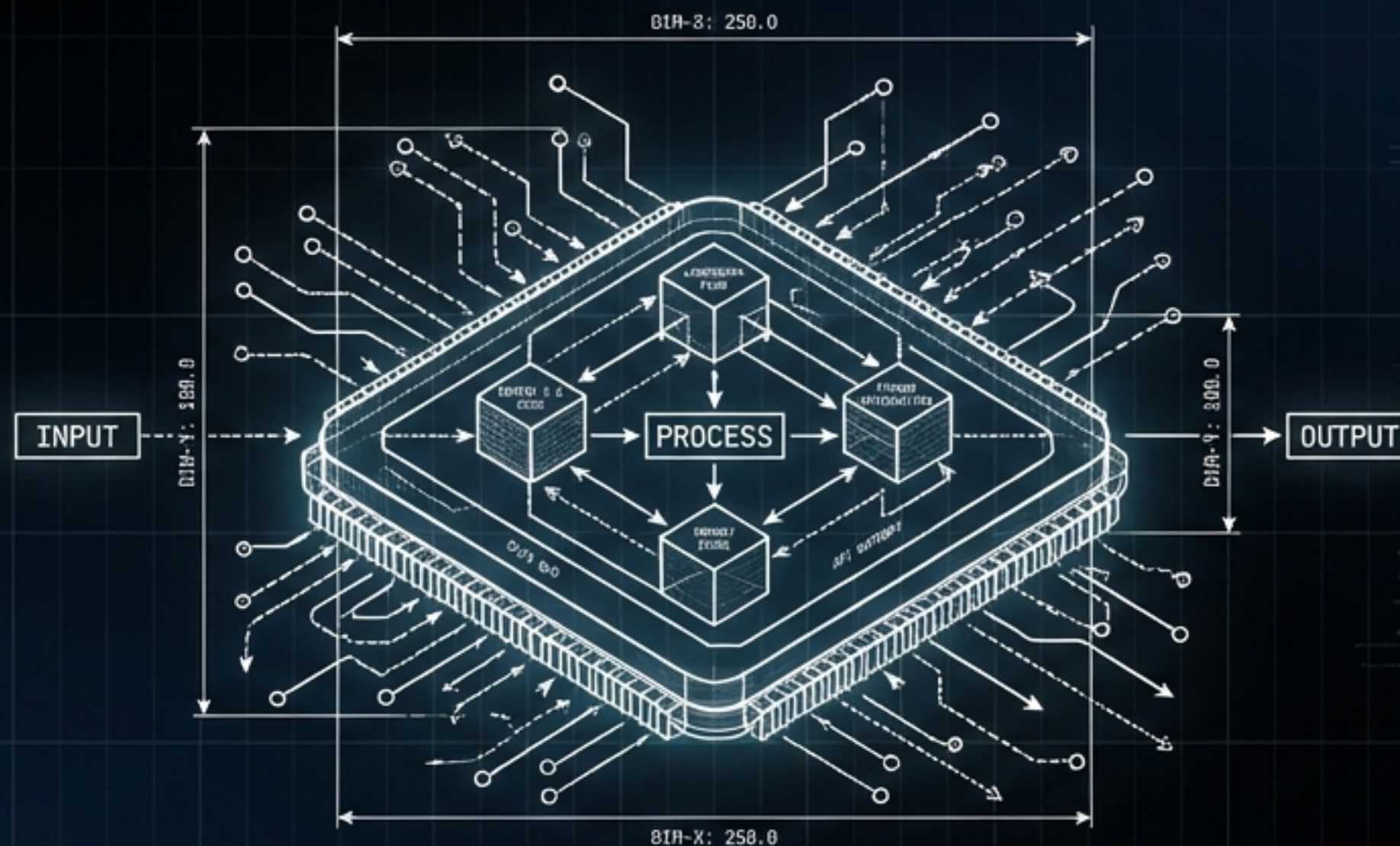


Google Gemini AI Agent 全栈开发实战指南

基于 Gemini 1.5、LangGraph 与 Python 的企业级智能体架构演进



从脚本到智能体：数字员工的进化路线图



“我们不是在写聊天机器人，而是在构建具备感知、决策和行动能力的数字员工。”

核心引擎与人设：赋予 AI 灵魂

Theory (原理)



Code (代码)

```
nomin.py x
python > reanin.py
1 model = genai.GenerativeModel(
2     model_name='models/gemini-1.5-pro',
3     system_instruction='你是一个资深的 Python 代码审查员。你的任务是'
4 )
5 chat = model.start_chat(history=[])
6
```

Ln 15, Col 1 Spaces: 4 UTF-8 LEF Python

原生多模态视觉：超越传统 OCR



```
{
  "invoice_number": "07312692",
  "date": "2021年04月25日",
  "seller_name": "上海浦东新区泰谷路65号 021-58668686",
  "seller_name": "上海浦东新区泰谷路65号 021-58668686",
  "buyer_name": "07312692",
  "total_amount": "719500.00",
  "items": [
    {
      "name": "融资租赁*首期租金",
      "amount": "636725.66",
      "tax": "82774.34"
    }
  ],
  "buyer_name": "哈密立强工程建设有限公司"
}
```

Native Vision: 直接解析图片语义 (文字、图表、Meme)

Visual JSON Mode: 非结构化图片 -> 严格结构化数据

连接真实世界：Function Calling 机制

Define (定义工具)

```
def function(ron):  
    """  
    Docstrings 是 AI 的说明书  
    """
```

```
@tool  
def get_exchange_rate(currency_from: str, currency_to: str):  
    '''查询两种货币之间的实时汇率'''  
    ...
```

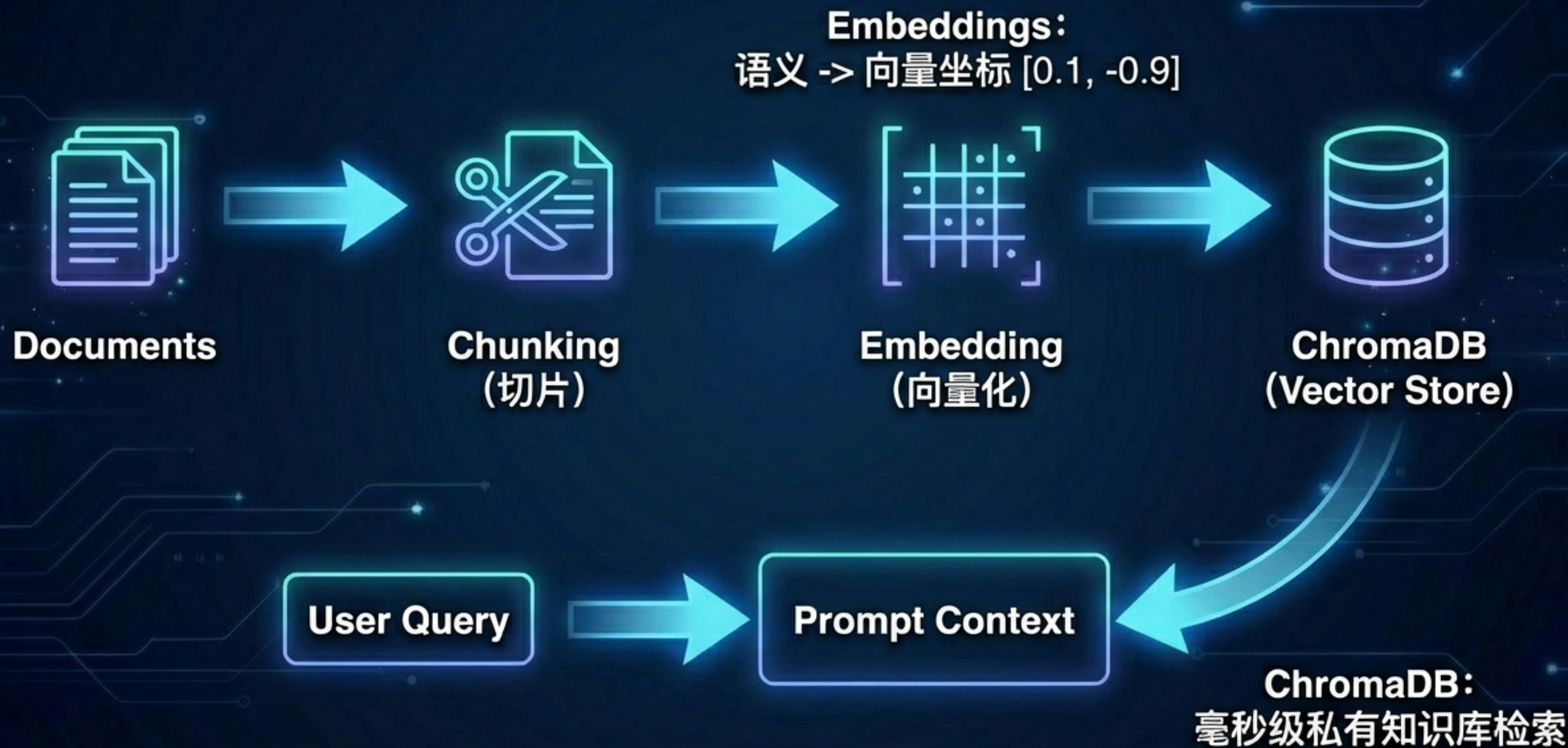
Observe (观察)
结果回填 Context

AI 不直接运行代码,
它只是发号施令

Decide (决策)
AI 输出 Call Request

Execute (执行)
本地 SDK 运行代码

突破知识截止：企业级 RAG 架构



上下文策略：RAG vs. 100万 Token 暴力美学

RAG (Retrieval-Augmented Generation)



机制：检索 Top-K 切片



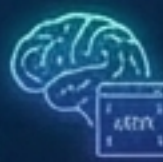
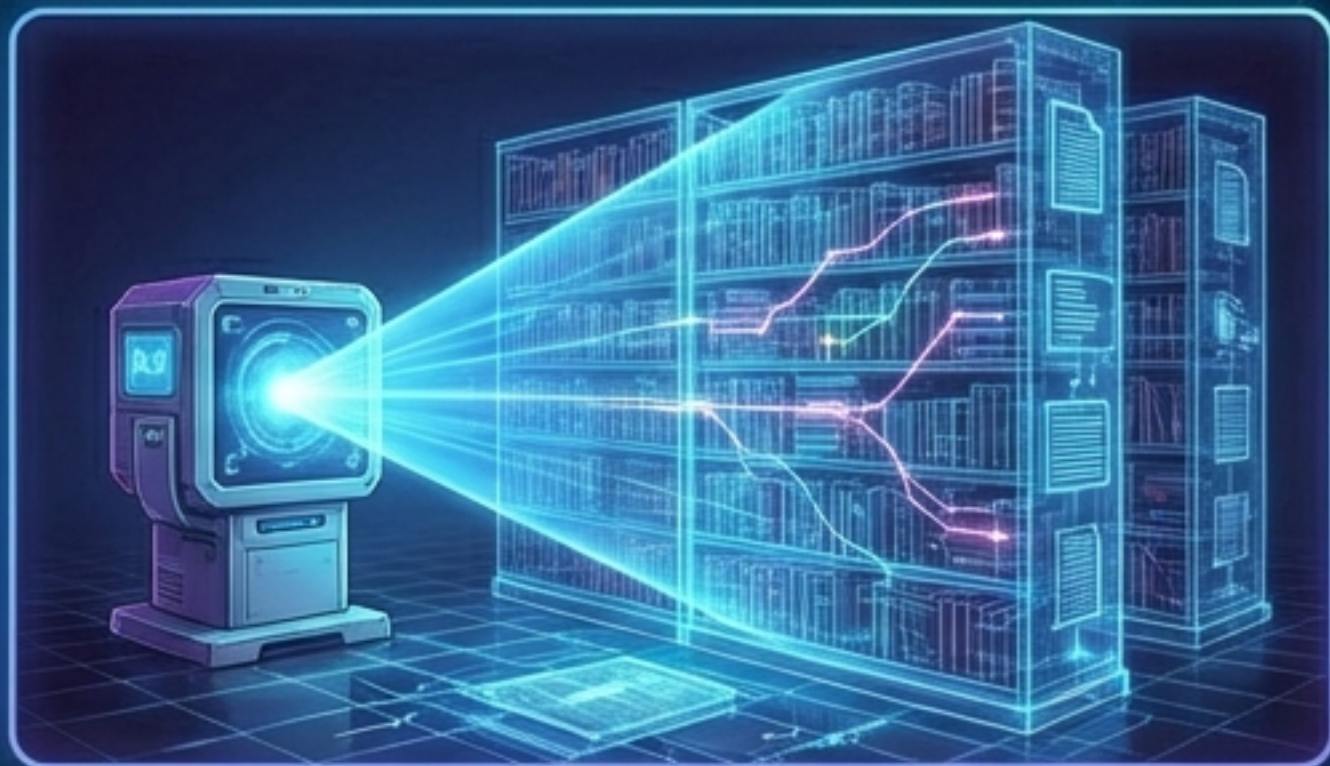
优点：成本低，速度快



场景：海量知识库查询 (Wiki)

Vs

Long Context (Gemini 1.5 Pro)



机制：全文放入 Context Window



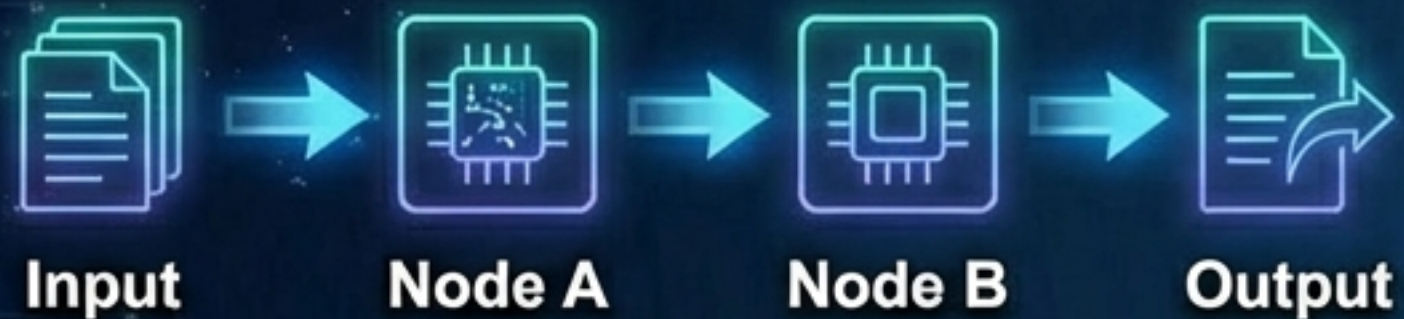
优点：跨段落推理，全局理解



场景：“大海捞针” (Needle in a Haystack) 测试

架构范式转移：从流水线（Chain）到状态机（Graph）

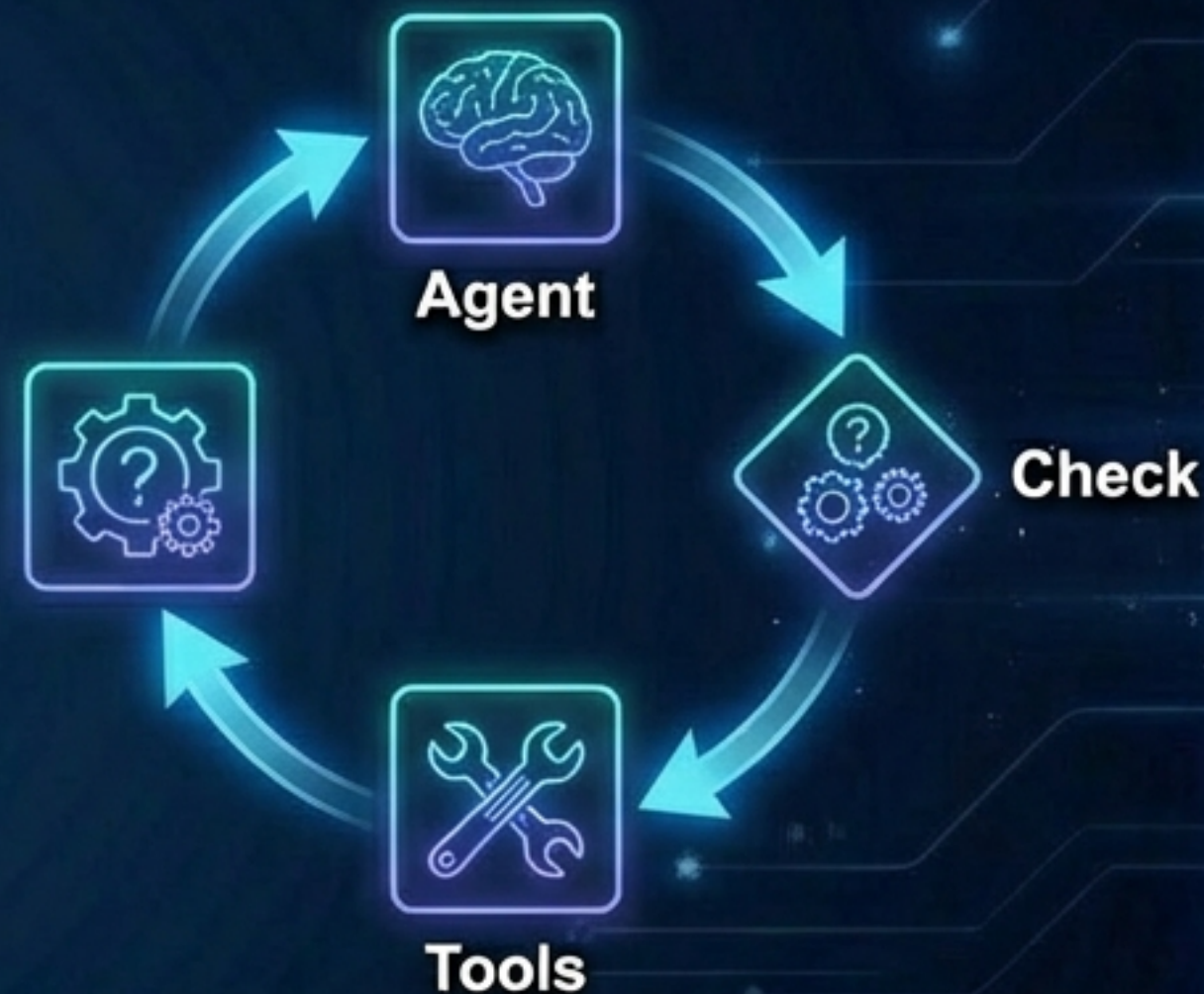
LangChain (DAG)



线性执行，适合单向任务

VS

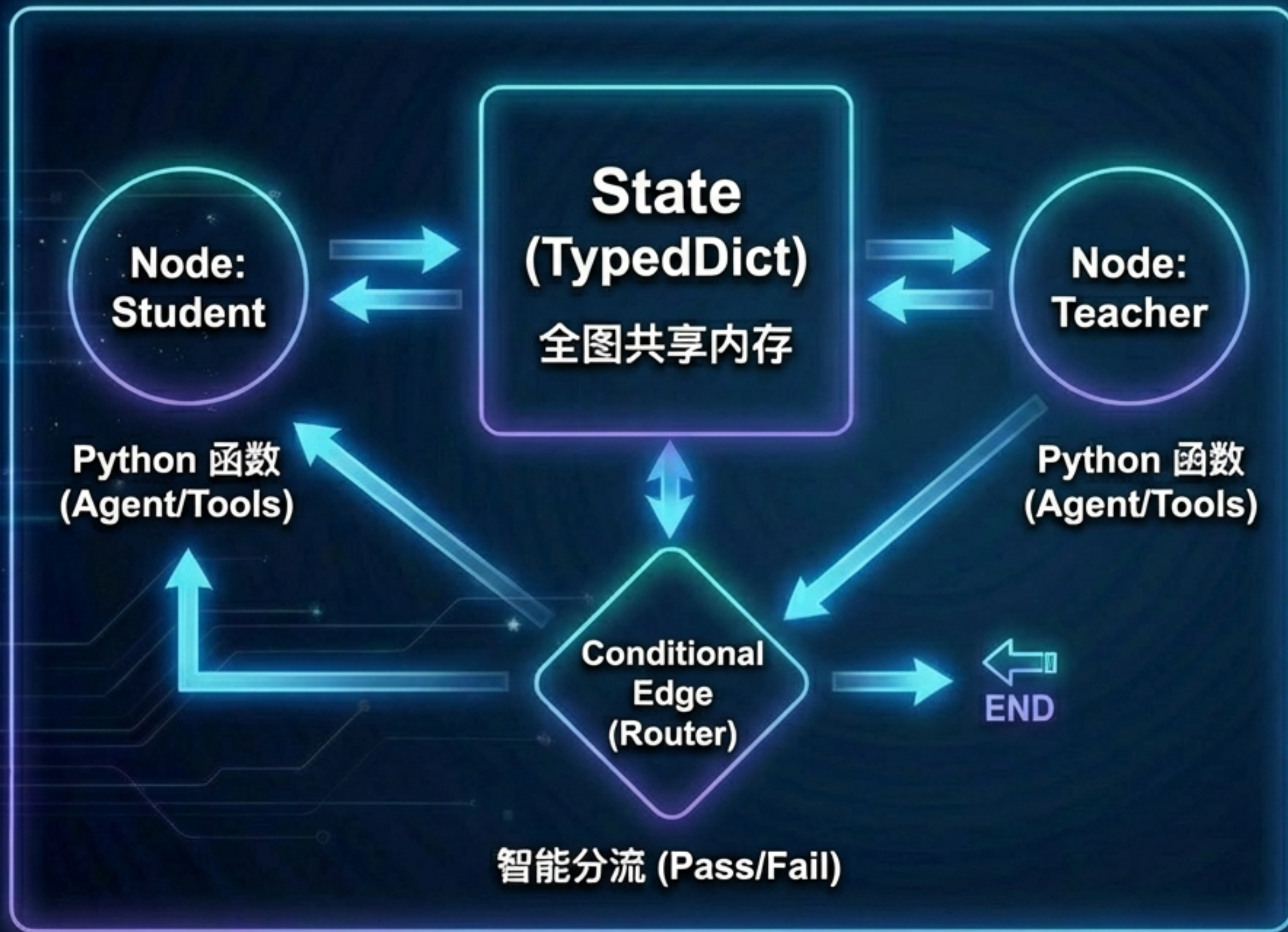
LangGraph (Cyclic)



循环结构支持自我修正 (Self-Correction)

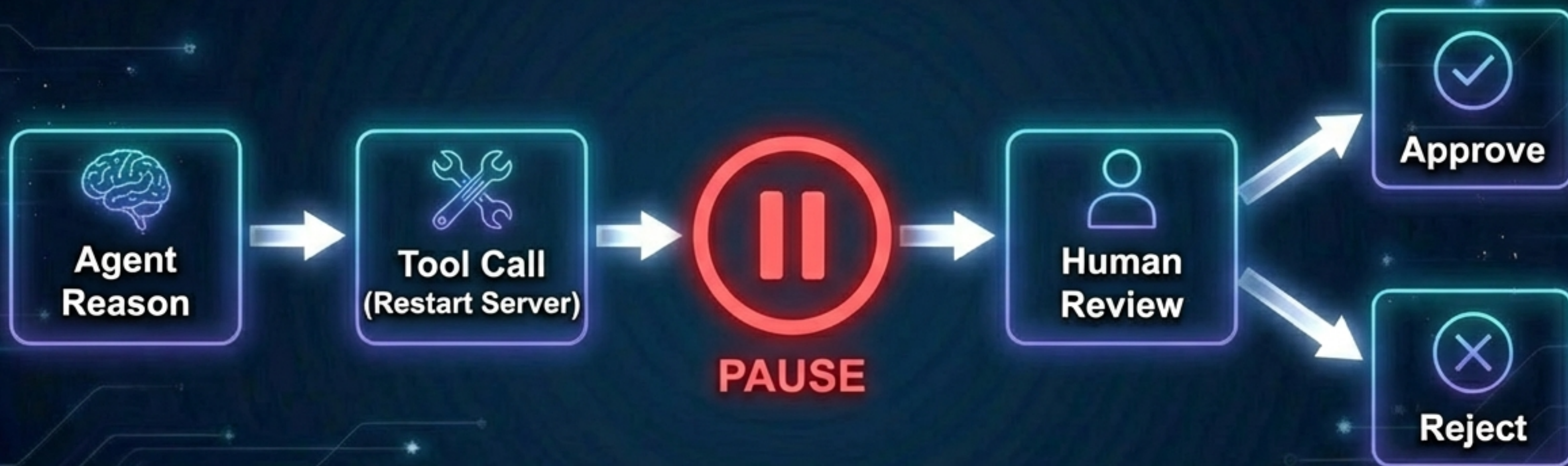
“Graph allows the agent to ‘think’ in a loop.”

编排思考过程：LangGraph 核心组件



```
workflow = StateGraph(MathState)
workflow.add_node('student', node_student)
workflow.add_conditional_edges(
    'teacher',
    router,
    {'pass': END, 'fail': 'student'}
)
```


安全阀门：Human-in-the-Loop (HITL)

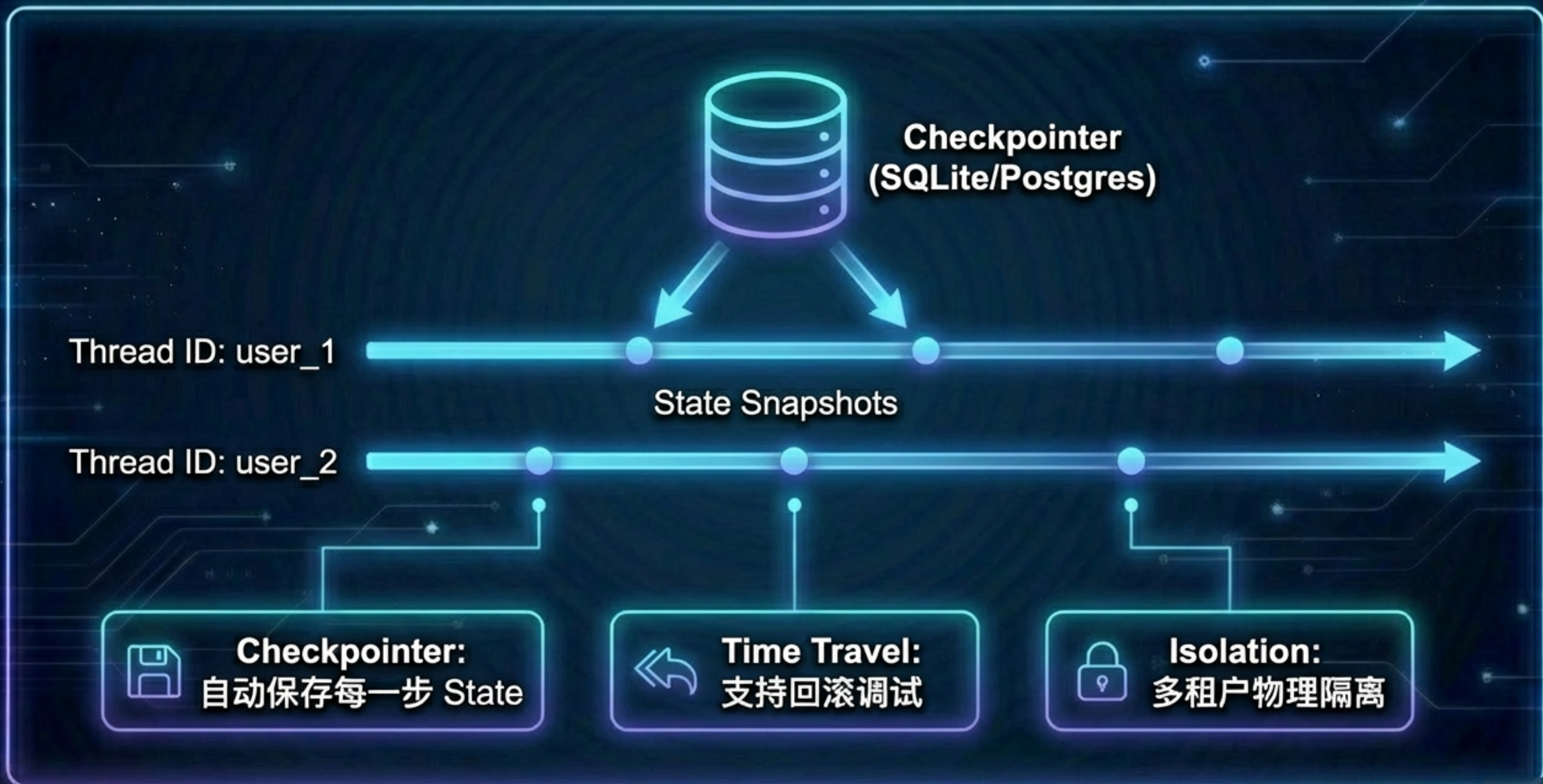


Scenario: 高危操作（重启服务器，转账）

Mechanism: interrupt_before=['tool_node']

Action: update_state（人类篡改决策） or resume（继续）

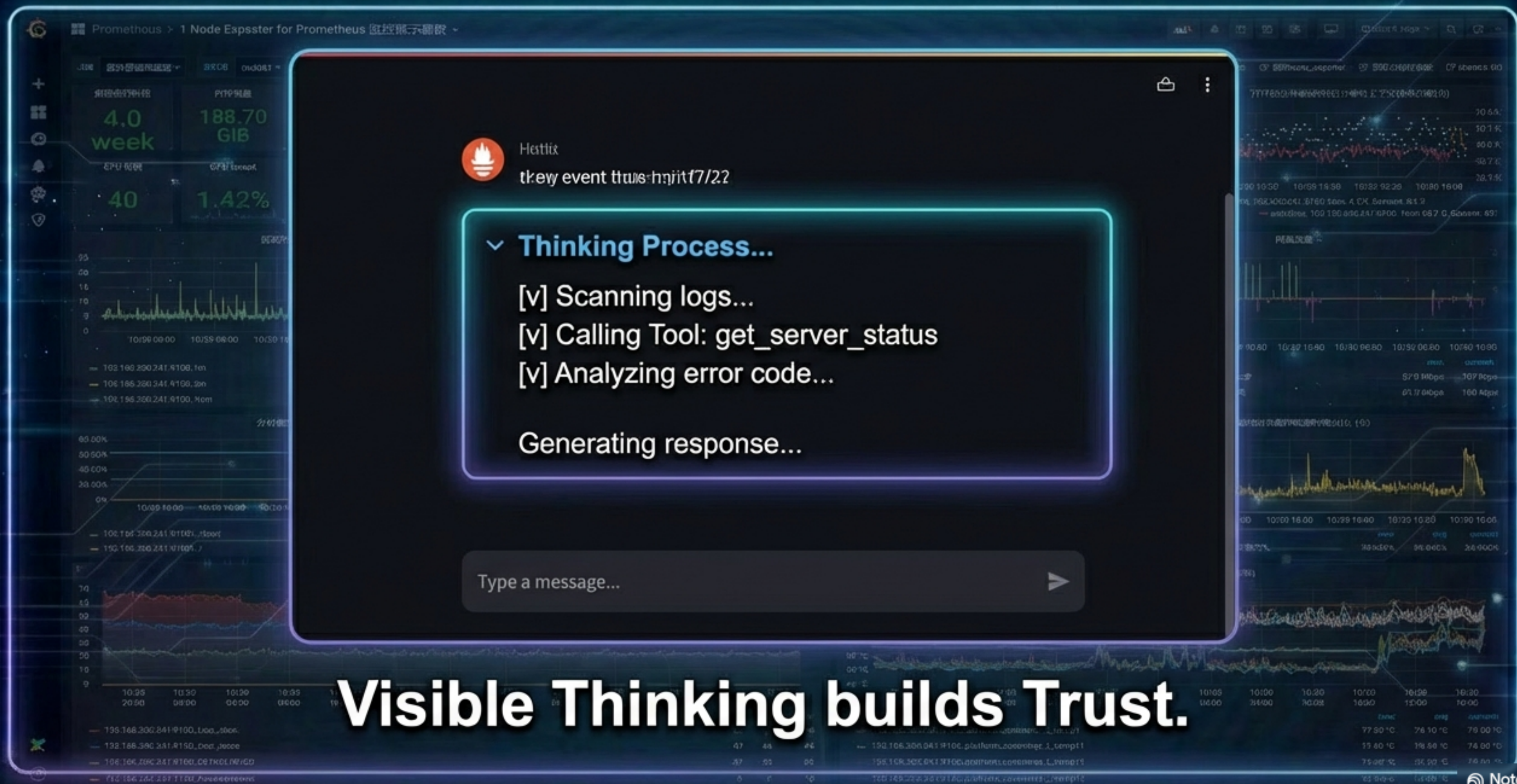
持久化与记忆隔离 (Persistence)



全栈工程化：Client-Server 微服务架构

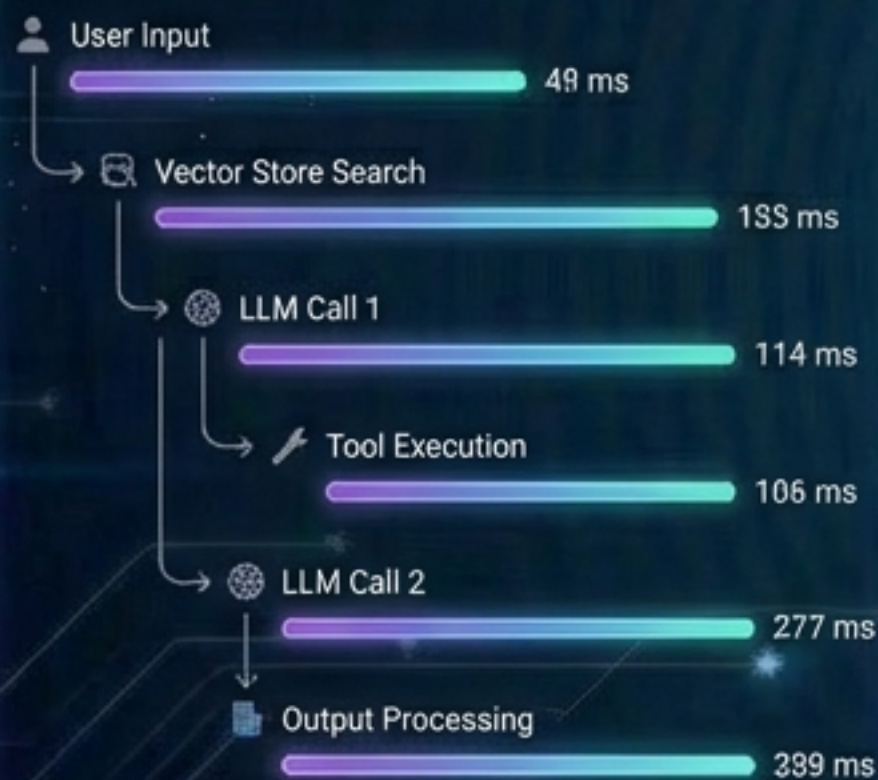


拒绝黑盒：思考过程可视化 (White-box UX)



系统免疫：LLMOps 与防御体系

Tracing (LangSmith)



Evaluation (LLM-as-a-Judge)

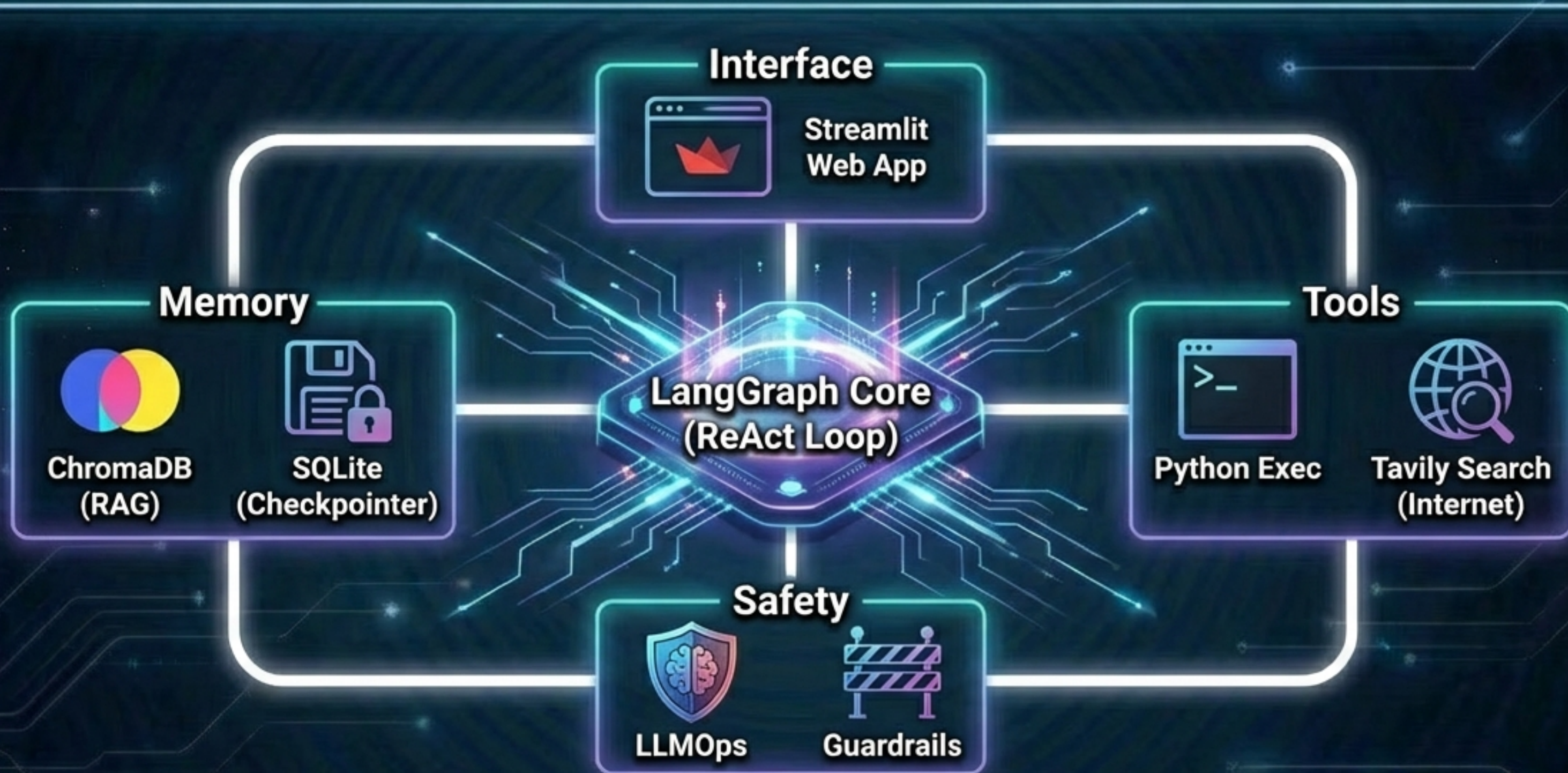


Guardrails



Retry Logic (Tenacity) -> Fallback (Pro to Flash) -> Safety Filter

终极形态：The Super Agent Architecture



“AI 不会取代开发者，但掌握 AI Agent 开发的开发者将取代那些不会的。”